



**RV College of
Engineering**

Go, change the world

Unit 3 (Relational Database Design)

Original Content:

Ramez Elmasri and Shamkant B. Navathe

Dr. Pratiba D

Assistant Professor, Department of CSE

RV College of Engineering, Bengaluru - 59

Contents

- 1 Informal Design Guidelines for Relational Databases
 - 1.1 Semantics of the Relation Attributes
 - 1.2 Redundant Information in Tuples and Update Anomalies
 - 1.3 Null Values in Tuples
 - 1.4 Spurious Tuples
- 2 Functional Dependencies (FDs)
 - 2.1 Definition of FD
 - 2.2 Inference Rules for FDs
 - 2.3 Equivalence of Sets of FDs
- 3 Normal Forms Based on Primary Keys
 - 3.1 Introduction to Normalization
 - 3.2 First Normal Form
 - 3.3 Second Normal Form
 - 3.4 Third Normal Form
- 4 General Normal Form Definitions (For Multiple Keys)
- 5 BCNF (Boyce-Codd Normal Form)

Informal Design Guidelines for Relational Databases

GUIDELINE 1: Informally, each tuple in a relation should represent one entity or relationship instance. (Applies to individual relations and their attributes).

- Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation
- Only foreign keys should be used to refer to other entities
- Entity and relationship attributes should be kept apart as much as possible.

Bottom Line: Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.

A Simplified Company Relational Schema

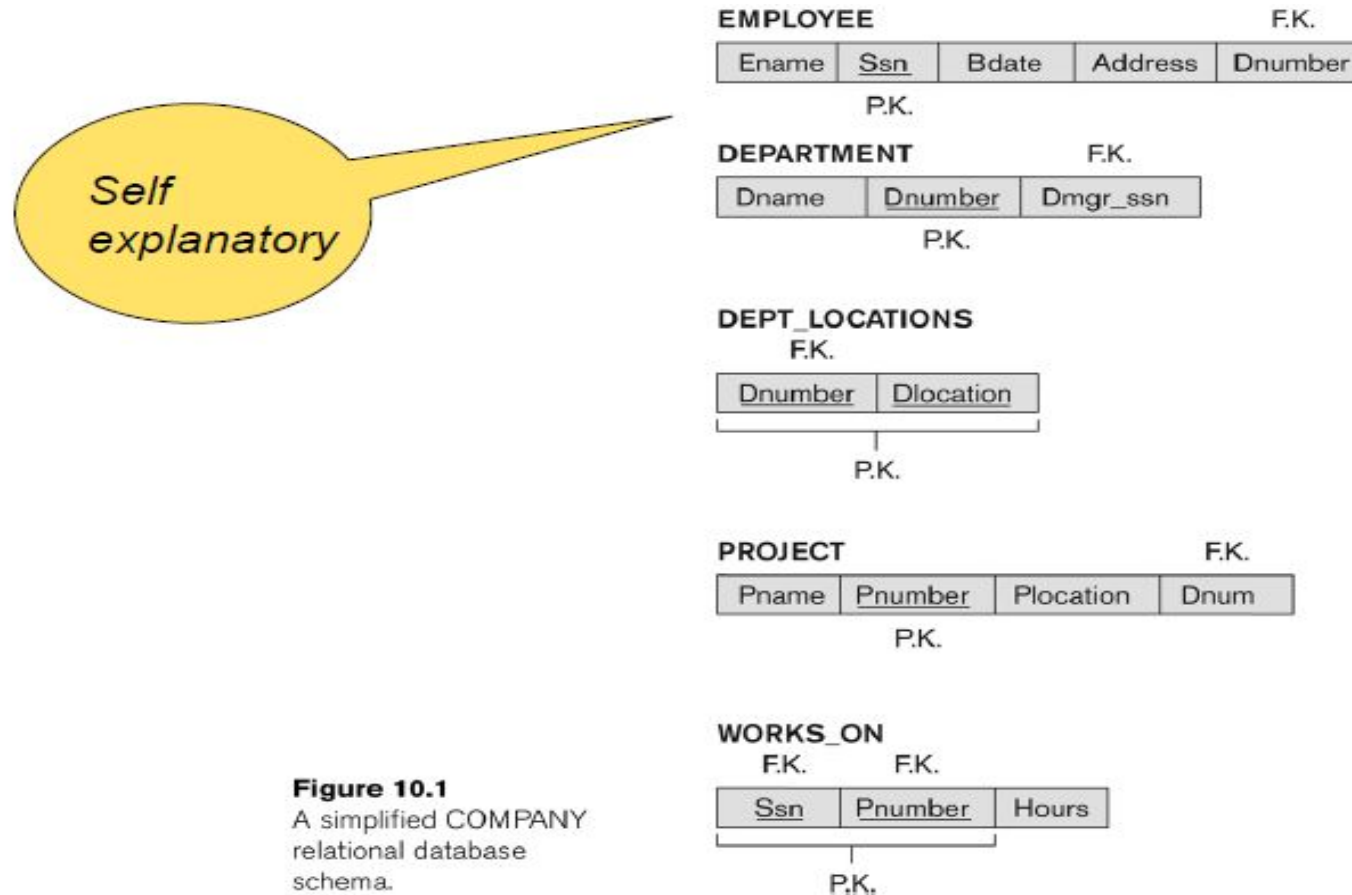


Figure 10.1
A simplified COMPANY
relational database
schema.

Redundant Information in Tuples and Update Anomalies

- **Big (and common) DB Problem:**

- In a poorly designed DB information is stored redundantly

- **Consequences:**

- Wastes storage
- Causes problems with update anomalies
 - Insertion anomalies
 - Deletion anomalies
 - Modification anomalies

EXAMPLE OF AN UPDATE ANOMALY

- Consider the relation:

EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)

- **Update Anomaly:**
 - Changing the name of project number P1 from "Billing" to "Customer-Accounting" may cause this update to be made for all 100 employees working on project P1.

EXAMPLE OF AN INSERT ANOMALY

- Consider the relation:

EMP_PROJ (Emp#, Proj#, Ename, Pname, No_hours)

- **Insert Anomaly:**
 - Cannot insert a project unless an employee is assigned to it.
- **Conversely**
 - Cannot insert an employee unless an he/she is assigned to a project.

Figure 10.3 Two relation schemas suffering from update anomalies

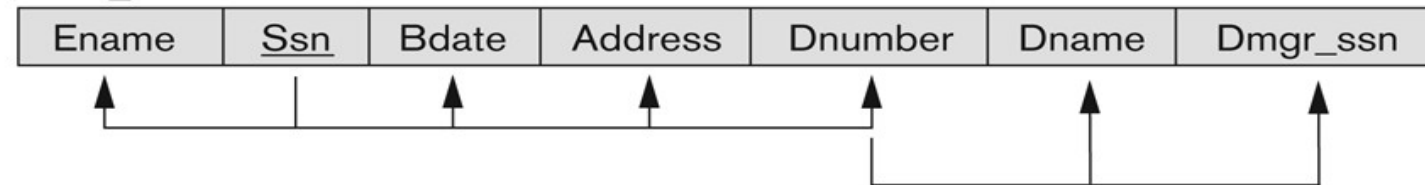
Figure 10.3

Two relation schemas suffering from update anomalies.

(a) EMP_DEPT and
(b) EMP_PROJ.

(a)

EMP_DEPT



(b)

EMP_PROJ

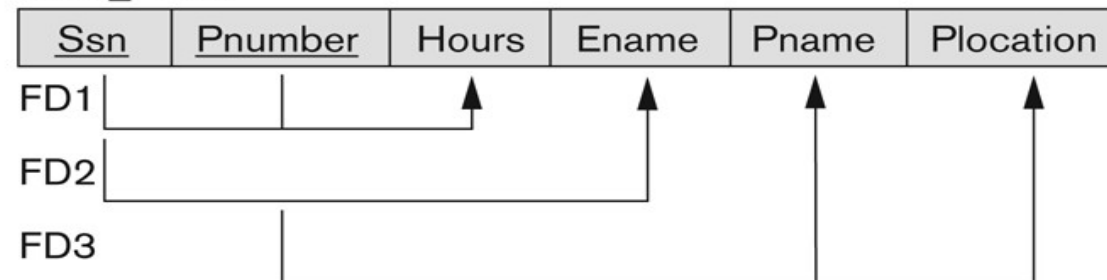


Figure 10.4 Example States for EMP_DEPT and EMP_PROJ

Figure 10.4

Example states for EMP_DEPT and EMP_PROJ resulting from applying NATURAL JOIN to the relations in Figure 10.2. These may be stored as base relations for performance reasons.

					Redundancy	
EMP_DEPT						
Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

EMP_PROJ			Redundancy		Redundancy	
Ssn	Pnumber	Hours	Ename	Pname	Plocation	
123456789	1	32.5	Smith, John B.	ProductX	Bellaire	
123456789	2	7.5	Smith, John B.	ProductY	Sugarland	
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston	
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire	
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland	
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland	
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston	
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford	
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston	
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford	
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford	
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford	
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford	
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford	
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston	
888665555	20	Null	Borg, James E.	Reorganization	Houston	

Guideline to Redundant Information in Tuples and Update Anomalies

■ **GUIDELINE 2:**

- Design a schema that does not suffer from the insertion, deletion and update anomalies.
- If there are any anomalies present, then note them so that applications can be made to take them into account.

Null Values in Tuples

■ GUIDELINE 3:

- Relations should be designed such that their tuples will have as few NULL values as possible
- Attributes that are NULL frequently could be placed in separate relations (with the primary key)

■ Reasons for nulls:

- Attribute not applicable or invalid
- Attribute value unknown (may exist)
- Value known to exist, but unavailable

Spurious Tuples

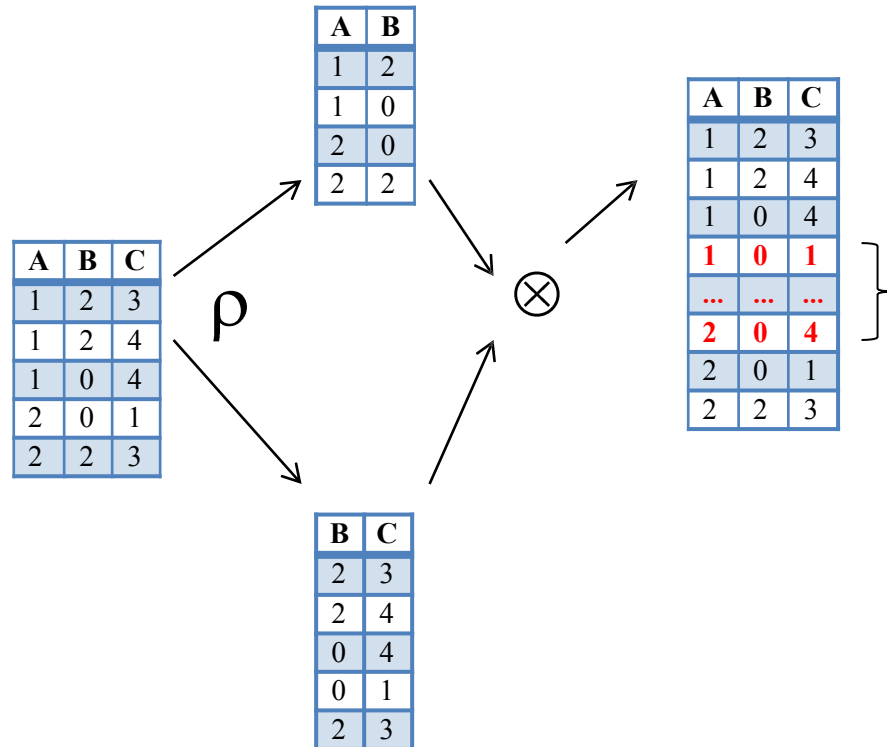
- Bad designs for a relational database may result in erroneous results for certain JOIN operations
- The "**lossless join**" property is used to guarantee meaningful results for join operations
- **GUIDELINE 4:**
 - The relations should be designed to satisfy the lossless join condition.
 - No spurious tuples should be generated by doing a natural-join of any relations.

Spurious Tuples (2)

- There are two important properties of decompositions:
 - a) Non-additive or losslessness of the corresponding join
 - b) Preservation of the functional dependencies.

- Note that:
 - Property (a) is extremely important and *cannot* be sacrificed.
 - Property (b) is less stringent and may be sacrificed. (See in next Chapter).

Spurious Tuples



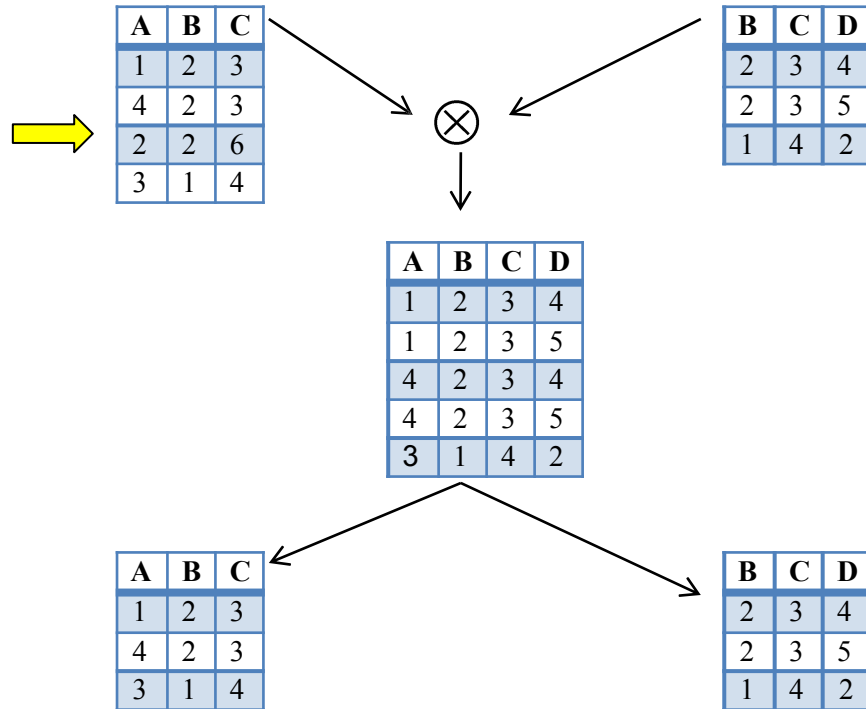
Consider relation $r(ABCD)$ and its projections $r1(AB)$ and $r2(BC)$.

Phantom records

Observation

Not all decompositions of a table can be combined using *natural join* to reproduce the original table.

Spurious Tuples



Consider the following two relations $r_1(ABC)$ and $r_2(BCD)$.

Compute natural join $r = r_1 r_2$

Evaluate projections $r_1' = \pi_{ABC}(r)$ and $r_2' = \pi_{BCD}(r)$

Observation

Tables r_2 and r_2' are the same however tuple $\langle 2, 2, 6 \rangle \in r_1'$ but not present in r_1

Functional Dependencies

- Functional dependencies (FDs) are used to specify *formal measures* of the "goodness" of relational designs
- FDs and keys are used to define **normal forms** for relations
- FDs are **constraints** that are derived from the *meaning* and *interrelationships* of the data attributes
- FD is a constraint between two sets of attributes
- A set of attributes *X functionally determines* a set of attributes Y if the value of X determines a unique value for Y

Functional Dependencies

- $X \twoheadrightarrow Y$ holds if whenever two tuples have the same value for X , they *must have* the same value for Y
- For any two tuples t_1 and t_2 in any relation instance $r(R)$:
$$\text{If } t_1[X]=t_2[X],$$
$$\text{then } t_1[Y]=t_2[Y]$$
- $X \twoheadrightarrow Y$ in R specifies a *constraint* on all relation instances $r(R)$
- Written as $X \twoheadrightarrow Y$; can be displayed graphically on a relation schema as in Figures.
(denoted by the arrow:).
- FDs are derived from the real-world constraints on the attributes

Example

TUPLE #	A	B	C
1	10	b1	c1
2	10	b2	c2
3	11	b4	c1
4	12	b3	c4
5	13	b1	c1
6	14	b3	c4

Does $A \rightarrow B$?

No.

$t_1[A] = t_2[A]$, but
 $t_1[B] \neq t_2[B]$

TUPLE #	A	B	C
1	10	b1	c1
2	10	b2	c2
3	11	b4	c1
4	12	b3	c4
5	13	b1	c1
6	14	b3	c4

Does $B \square C$

Examples of FD constraints

TUPLE
#

	A	B	C
1	10	b1	c1
2	10	b2	c2
3	11	b4	c1
4	12	b3	c4
5	13	b1	c1
6	14	b3	c4

Does $B \rightarrow C$?

Yes!

Look at tuples t_1
and t_5 , and tuples
 t_4 and t_6

TUPLE
#

	A	B	C
1	10	b1	c1
2	10	b2	c2
3	11	b4	c1
4	12	b3	c4
5	13	b1	c1
6	14	b3	c4

Does $C \rightarrow B$?

No.

$t_1[C] = t_3[C]$, but
 $t_1[B] \neq t_3[B]$

Examples of FD constraints

- social security number determines employee name
SSN $\rightarrow$ ENAME
- project number determines project name and location
PNUMBER $\rightarrow$ {PNAME, PLOCATION}
- employee ssn and project number determines the hours per week that the employee works on the project
{SSN, PNUMBER} $\rightarrow$ HOURS

Examples of FD constraints

- An FD is a property of the attributes in the schema R
- The constraint must hold on *every relation instance* $r(R)$
- If K is a key of R , then K functionally determines all attributes in R (since we never have two distinct tuples with $t1[K]=t2[K]$)

FDs must hold for all valid states of a relation, not just current state

- So define FDs carefully!

How do we identify FDs?

- Likely, some FDs will be obvious or identified in initial design of DB

Vehicle(tagno, regstate, owner, make, model, year, gaseconomy, dealership, dealeraddr)

{tagno, regstate} → owner

{tagno, regstate} → {make, model, year}

{make, model, year} → gaseconomy,

dealership → dealeraddr etc...

How do we identify FDs?

- But the algorithms we use to test for other properties of good DB design often need to know **ALL** FDs!
- Some FDs may not be obvious, but can be deduced from other FDs
- Given a set of FDs F for a relation R , the set of all FDs for R is **F^+**
(known as the ***closure of F***)

Inferring FDs

Ex: (SSN, PNUMBER, HOURS, ENAME, PNAME,
PLOCATION)

SSN $\rightarrow$ ENAME,
{SSN, PNUMBER} $\rightarrow$ HOURS, PNUMBER $\rightarrow$
PNAME, PNUMBER $\rightarrow$ PLOCATION

PNUMBER $\rightarrow$ PNAME,
so {PNUMBER, HOURS} $\rightarrow$ PNAME

PNUMBER $\rightarrow$ PNAME and PNUMBER $\rightarrow$ PLOCATION, so PNUMBER $\rightarrow$
{PNAME, PLOCATION}

Inference Rules for FDs

- Given a set of FDs F , we can *infer* additional FDs that hold whenever the FDs in F hold

Armstrong's inference rules:

IR1. (**Reflexive**) If Y subset-of X , then $X \rightarrow Y$

IR2. (**Augmentation**) If $X \rightarrow Y$, then $XZ \rightarrow YZ$

(Notation: XZ stands for $X \cup Z$)

IR3. (**Transitive**) If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

- IR1, IR2, IR3 form a *sound* and *complete* set of inference rules

Inference Rules for FDs

Some additional inference rules that are useful:

(Decomposition) If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$

(Union) If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$

(Pseudotransitivity) If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$

- The last three inference rules, as well as any other inference rules, can be deduced from IR1, IR2, and IR3 (completeness property)

Inference Rules for FDs

- **Closure** of a set F of FDs is the set F^+ of all FDs that can be inferred from F
- **Closure** of a set of attributes X with respect to F is the set X^+ of all attributes that are functionally determined by X
- X^+ can be calculated by repeatedly applying IR1, IR2, IR3 using the FDs in F

Closure of X under F (X^+)

- X^+ = set of all attributes dependent on X
- Algorithm
 1. start with $X^+ = X$
 2. for each FD $Y \rightarrow Z$ in F do
 - if Y is a subset of X^+ then $X^+ = X^+ \cup Z$
 3. Continue this process until no more attributes can be added to X^+

Example

Given a relation Student and a set of functional dependencies F as follows, compute the closure for all LHS.

Student(SID, dept, dept_chair)

$F = \{ \text{SID} \rightarrow \{\text{dept}, \text{dept_chair}\}, \text{dept} \rightarrow \text{dept_chair},$
 $\{\text{SID}, \text{dept}\} \rightarrow \text{dept_chair} \}$

$\{\text{SID}\}^+ = \{\text{SID}, \text{dept}, \text{dept_chair}\}$

$\{\text{dept}\}^+ = \{\text{dept}, \text{dept_chair}\}$

$\{\text{SID}, \text{dept}\}^+ = \{\text{SID}, \text{dept}, \text{dept_chair}\}$

If the closure of a LHS includes all attributes, then this LHS is a **super key** of the relation.

Candidate Keys

- If X^+ contains all attributes in a relation R , and if there **does not exist** Y in X such that $(X - Y)^+ = \text{all attributes in } R$, then X is a **candidate key** for R
- In previous example, $\{\text{SID}, \text{dept}\}^+$ includes all attributes, SID^+ also includes all attributes.
- **SID** is a candidate key

Exercise

$R(A, B, C, D, G, H)$

$F = \{ A \rightarrow B, B \rightarrow C, CD \rightarrow H, BC \rightarrow G \}$

- What is the closure of AC?

Equivalence of Sets of FDs

- Two sets of FDs F and G are **equivalent** if:
 - every FD in F can be inferred from G , *and*
 - every FD in G can be inferred from F
- Hence, F and G are equivalent if $F^+ = G^+$

Definition: F **covers** G if every FD in G can be inferred from F (i.e., if G^+ subset-of F^+)

- F and G are equivalent if F covers G and G covers F
- There is an algorithm for checking equivalence of sets of FDs

Example of Equivalent FD sets

$F1 = \{ \text{SID} \rightarrow \{\text{dept}, \text{dept_chair}\}, \text{dept} \rightarrow \text{dept_chair},$
 $\{\text{SID}, \text{dept}\} \rightarrow \text{dept_chair} \}$

$F2 = \{ \text{SID} \rightarrow \{\text{dept}, \text{dept_chair}\}, \text{dept} \rightarrow \text{dept_chair} \}$

- F1 covers F2
- Only need to check if F2 can also infer F1
 - i.e. Can we generate the set of FDs in F1 using FDs defined in F2?
- Compute $\{\text{SID}, \text{dept}\}^+$ based on F2
 - Its closure includes all attributes
 - We can conclude that $\{\text{SID}, \text{dept}\} \rightarrow \text{dept_chair}$

Minimal Sets of FDs

- A set of FDs is **minimal** if it satisfies the following conditions:
 - (1) Every dependency in F has a single attribute for its RHS.
 - (2) We cannot remove any dependency from F and have a set of dependencies that is equivalent to F .
 - (3) We cannot replace any dependency $X \rightarrow A$ in F with a dependency $Y \rightarrow A$, where Y proper-subset-of X (Y subset-of X) and still have a set of dependencies that is equivalent to F .

Minimal Sets of FDs

- Every set of FDs has an equivalent minimal set
- There can be several equivalent minimal sets
- There is no simple algorithm for computing a minimal set of FDs that is equivalent to a set F of FDs

Algorithm

Given a set of FDs F , find its minimal cover

- Step1: Decompose each FD to get single attribute at **RHS**
- Step2: For each FD, remove redundant attribute from **LHS**
- Step3: Remove redundant **FDs**

Example

Given $F = \{B \rightarrow AB, D \rightarrow A, AB \rightarrow D\}$

- Step 1: $B \rightarrow AB$ is decomposed into $B \rightarrow A, B \rightarrow B$
($B \rightarrow B$ is trivial and is removed)
- Step 2: check if $AB \rightarrow D$ has redundant LHS. Can it be $A \rightarrow D$ or $B \rightarrow D$?

Compute AB^+, A^+, B^+ based on **F**

$AB^+ = ABD$ and $B^+ = ABD$, so A is extraneous.

- So far, we have $F = \{B \rightarrow A, D \rightarrow A, B \rightarrow D\}$

Example (cont.)

- So far, we have $F = \{B \rightarrow A, D \rightarrow A, B \rightarrow D\}$
- Step 3: check if there is any redundant FDs. Is $B \rightarrow A$ redundant?

Compute B^+ based on $F - \{B \rightarrow A\}$

$B^+ = BDA$, that means we can obtain

$B \rightarrow A$ from $F - \{B \rightarrow A\}$ so $B \rightarrow A$ is redundant.

Similarly, check the remaining FDs.

Final answer $F' = \{D \rightarrow A, B \rightarrow D\}$

Exercise

Given a set of FDs F , find its **minimal cover**

- Step1: Decompose each FD to get single attribute at RHS
- Step2: For each FD, remove redundant attribute from LHS
- Step3: Remove redundant FDs

Question: what is the minimal cover of F ?

$R(A, B, C), F = \{A \rightarrow B, BC \rightarrow A, AB \rightarrow AC\}$

Minimal Set (Cover) of FDs

- There can be more than one minimal cover for a relation
- They won't necessarily have **the same number of FDs**

Normalization of Relations

- **Normalization:** The process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations
- **Normal form:** Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

Normalization of Relations

- **2NF, 3NF, BCNF**
 - based on keys and FDs of a relation schema
- **4NF**
 - based on keys, multi-valued dependencies : MVDs; 5NF based on keys,
 - Join dependencies : JDs (Chapter 11)
- Additional properties may be needed to ensure a good relational design (*lossless join, dependency preservation*; Chapter 11)

Practical Use of Normal Forms

- **Normalization** is carried out in practice so that the resulting designs are of high quality and meet the desirable properties
- The practical utility of these normal forms is questionable when the constraints on which they are based are *hard to understand* or to *detect*
- The database designers *need not* normalize to the highest possible normal form
 - (usually up to 3NF, BCNF or 4NF)
- **Denormalization:**
 - The process of storing the join of higher normal form relations as a base relation—which is in a lower normal form

Definitions of Keys and Attributes Participating in Keys

- A **superkey** of a relation schema $R = \{A_1, A_2, \dots, A_n\}$ is a set of attributes S *subset-of* R with the property that no two tuples t_1 and t_2 in any legal relation state r of R will have $t_1[S] = t_2[S]$
- A **key** K is a superkey with the *additional property* that removal of any attribute from K will cause K not to be a superkey any more.

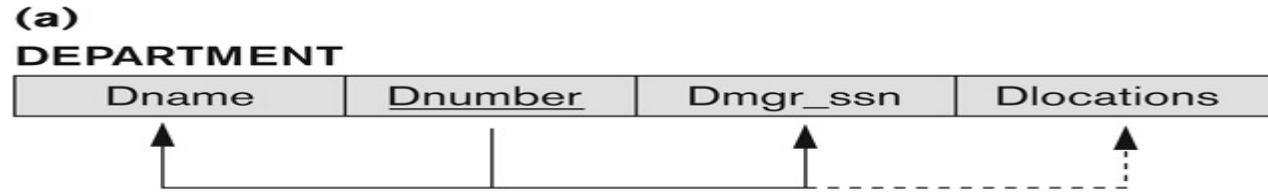
Definitions of Keys and Attributes Participating in Keys

- If a relation schema has more than one key, each is called a **candidate key**.
 - One of the candidate keys is *arbitrarily* designated to be the **primary key**, and the others are called **secondary keys**.
- A **Prime attribute** must be a member of *some* candidate key
- A **Nonprime attribute** is not a prime attribute—that is, it is not a member of any candidate key.

First Normal Form 1NF

- A relation scheme R is in first normal form (1NF) if the values in $dom(A)$ are atomic for every attribute A in R .
- **Disallows**
 - composite attributes
 - Set-valued attributes
 - **nested relations**; a cell of an *individual tuple* is a complex relation

First Normal Form 1NF



(b)
DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)
DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Figure 10.8
Normalization into 1NF.
(a) A relation schema that is not in 1NF. (b) Example state of relation DEPARTMENT. (c) 1NF version of the same relation with redundancy.

Normalization nested relations into 1NF

(a)

EMP_PROJ

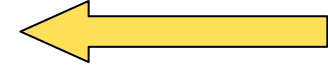
Ssn	Ename	Projs	
		Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, AliciaJ.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

Composite
attributes



(c)

EMP_PROJ1

Ssn	Ename
-----	-------

EMP_PROJ2

Ssn	Pnumber	Hours
-----	---------	-------

Figure 10.9
Normalizing nested relations into 1NF. (a) Schema of the EMP_PROJ relation with a *nested relation* attribute PROJS. (b) Example extension of the EMP_PROJ relation showing nested relations within each tuple. (c) Decomposition of EMP_PROJ into relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.

Second Normal Form

- Uses the concepts of **FDs, primary key**

- **Definitions**

- **Prime attribute:** An attribute that is member of the primary key K
- **Left-Reduced or Full functional dependency:** a FD $Y \twoheadrightarrow Z$ where removal of any attribute from Y means the FD does not hold any more

- **Examples:**

- $\{SSN, PNUMBER\} \twoheadrightarrow HOURS$ is a full FD since neither $SSN \twoheadrightarrow HOURS$ nor $PNUMBER \twoheadrightarrow HOURS$ hold
- $\{SSN, PNUMBER\} \twoheadrightarrow ENAME$ is not a full FD (it is called a partial dependency) since $SSN \twoheadrightarrow ENAME$ also holds

Second Normal Form

- A relation scheme R is in second normal form (**2NF**) with respect to a set of FDs F if it is in 1NF and every nonprime attribute is fully dependent on every key of R .
 - R can be decomposed into 2NF relations via the process of 2NF normalization
- **Example**
 - Let $R=ABCD$ and $F = \{ AB \rightarrow C, B \rightarrow D \}$. Here AB is a key. C and D are non-prime. C is fully dependent on the entire key AB , however D functionally depends on just *part* of the key ($B \rightarrow D$). This is called a *partial dependency*

Second Normal Form

Example

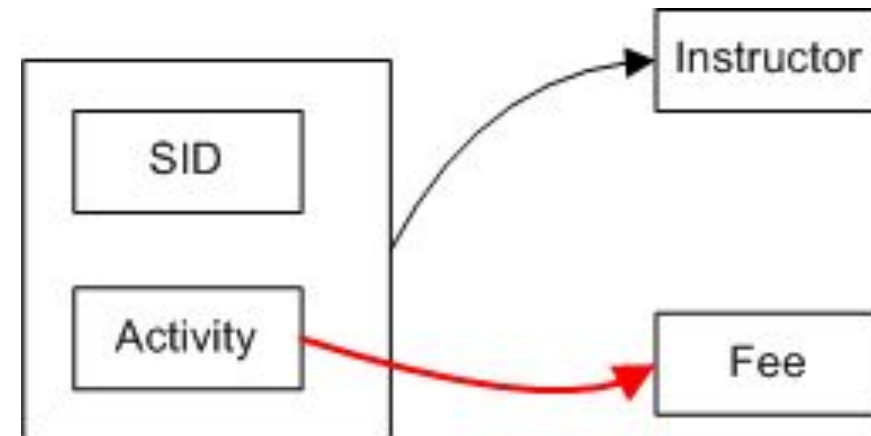
We deduce from the data sample

activity $\rightarrow$ fee,
sid activity $\rightarrow$ instructor

SID	Activity	Fee	Instructor
100	Basket Ball	200	Lebron
100	Golf	65	Arnold
200	Golf	65	Jack
300	Golf	65	Lebron

Key: {sid ,activity}. Non-key attributes: { Fee, Instructor }

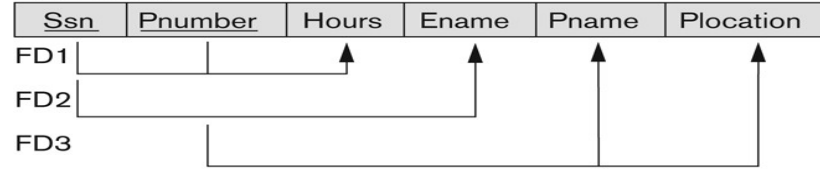
There is a partial dependency
therefore the schema is not 2NF



Normalizing into 2NF and 3NF

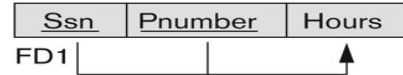
(a)

EMP_PROJ

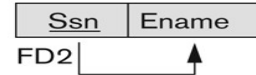


2NF Normalization

EP1



EP2



EP3

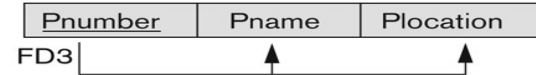
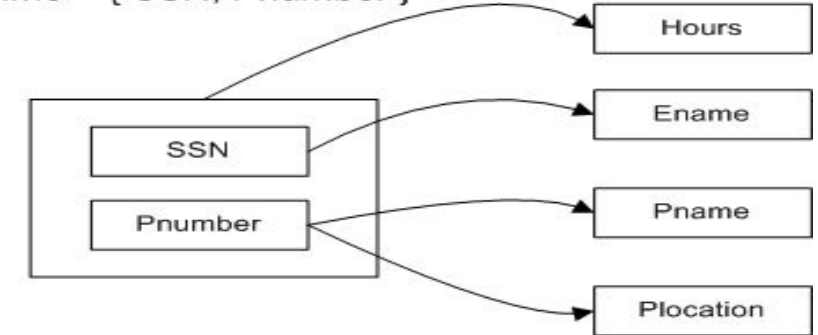


Figure 10.10

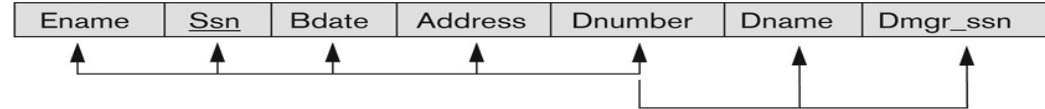
Normalizing into 2NF and 3NF.
(a) Normalizing EMP_PROJ into 2NF
relations. (b) Normalizing EMP_DEPT
into 3NF relations.

Prime = { SSN, Pnumber }



(b)

EMP_DEPT



3NF Normalization

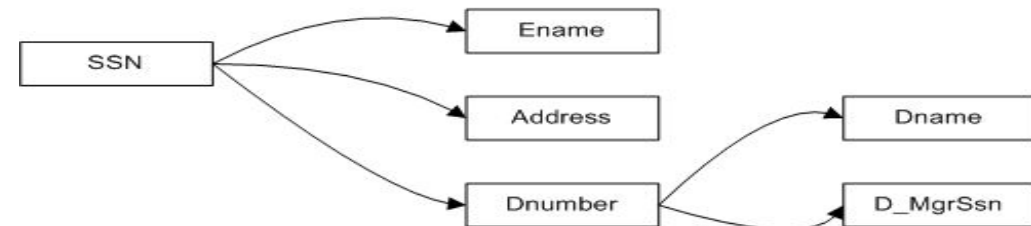
ED1



ED2



Prime = { SSN }



Normalization into 2NF and 3NF

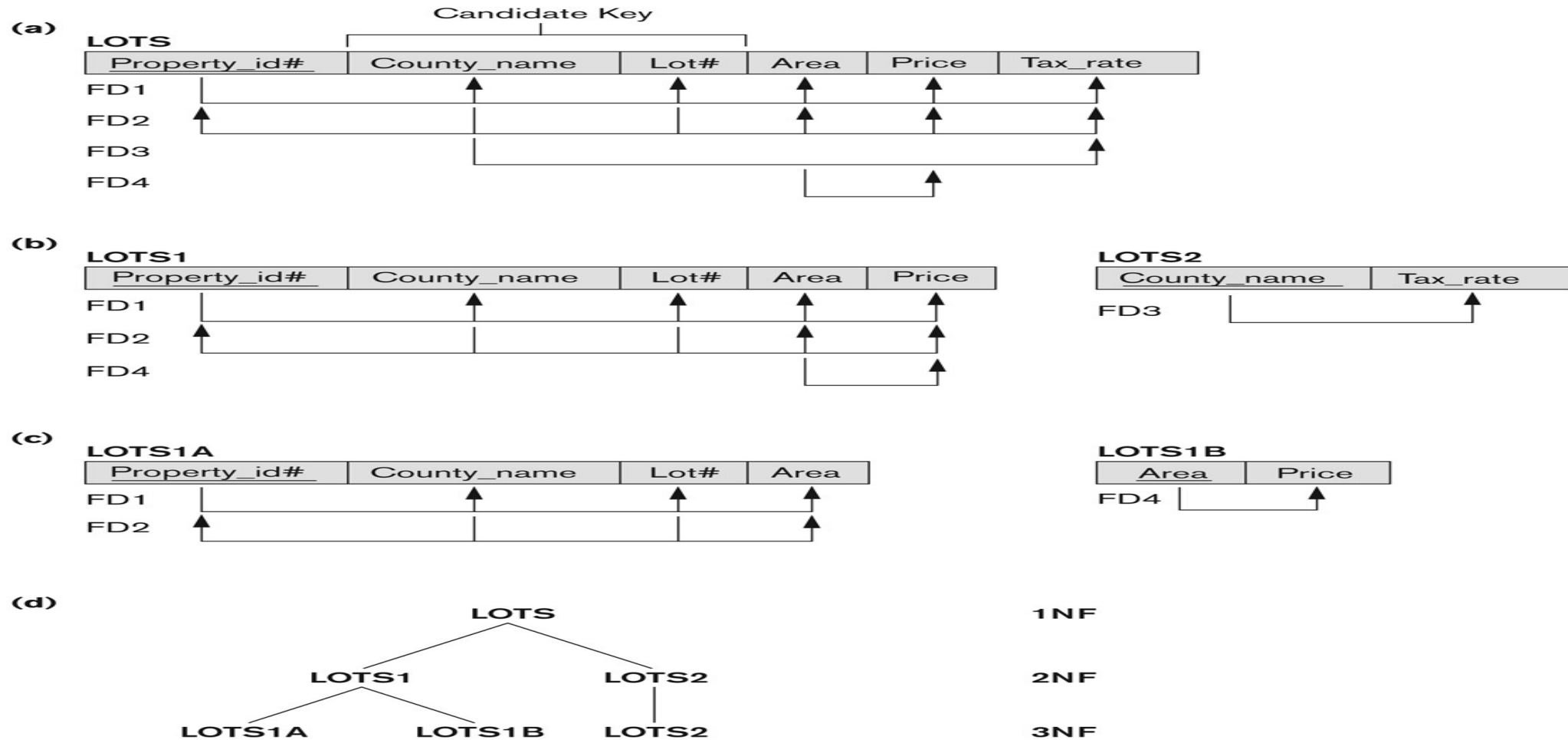


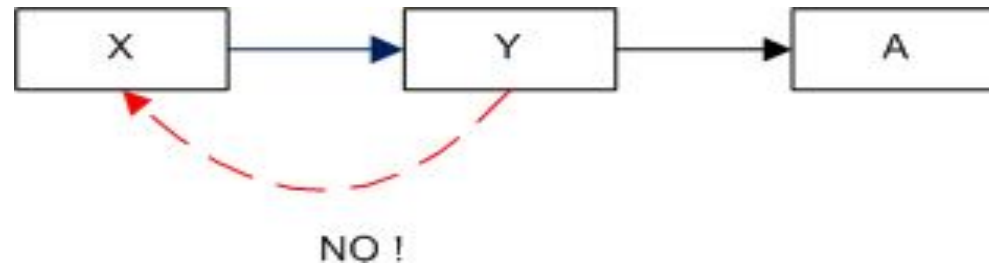
Figure 10.11 Normalization into 2NF and 3NF. (a) The LOTS relation with its functional dependencies FD1 through FD4. (b) Decomposing into the 2NF relations LOTS1 and LOTS2. (c) Decomposing LOTS1 into the 3NF relations LOTS1A and LOTS1B. (d) Summary of the progressive normalization of LOTS.

Third Normal Form

■ Definition:

Given a relation scheme R , a subset X of R , an attribute A in R , and a set of FDs F , A is **transitively dependent** upon X in R if there is a subset Y of R with:

$X \twoheadrightarrow Y$, $Y \not\rightarrow X$ and $Y \twoheadrightarrow A$ under F and $A \notin XY$.



Examples:

Schema $(ABCD)$ and $F = \{A \twoheadrightarrow B, B \twoheadrightarrow AC, C \twoheadrightarrow D\}$

D is transitively dependent on A (and B) via C , however C is not transitively dependent on A via B (B is prime).

Third Normal Form

- A relation schema R is in **third normal form (3NF)** if it is in 2NF *and* no non-prime attribute A in R is transitively dependent on the primary key
- R can be decomposed into 3NF relations via the process of 3NF normalization
- **NOTE:**
 - In $X \twoheadrightarrow Y$ and $Y \twoheadrightarrow Z$, with X as the primary key, we consider this a problem only if Y is not a candidate key.
 - When Y is a candidate key, there is no problem with the transitive dependency .
 - E.g., Consider EMP (SSN, Emp#, Salary).
 - Here, $SSN \twoheadrightarrow Emp\# \twoheadrightarrow Salary$ and Emp# is a candidate key.

Normal Forms Defined Informally

- 1st normal form
 - All attributes depend on **the key**
- 2nd normal form
 - All attributes depend on **the whole key**
- 3rd normal form
 - All attributes depend on **nothing but the key**

General Normal Form Definitions

- The above definitions consider the primary key only
- The following more general definitions take into account relations with multiple candidate keys

A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on *every* key of R

General Normal Form Definitions

- **Example** Consider the schema
- **SUPPLIER(sname, saddress, item, iname, price) and FDs**
- **$F = \{ \text{sname} \twoheadrightarrow \text{saddress}, \text{item} \twoheadrightarrow \text{iname}, \{\text{sname}, \text{item}\} \twoheadrightarrow \text{price} \}$**
- 1. *sname item* is the primary key, all other attributes are non-prime.
- 2. Observe that *saddress* depends on part of the key (*sname*).
- 3. Likewise *iname* depends on part of the key (*item*)
- 4. Therefore SUPPLIER is not in 2NF

General Normal Form Definitions

- Definition:
 - **Superkey** of relation schema R - a set of attributes S of R that contains a key of R
- A relation schema R is in **third normal form (3NF)** if whenever a FD $X \twoheadrightarrow A$ holds in R , then either:
 - (a) X is a superkey of R , or
 - (b) A is a prime attribute of R

General Normal Form Definitions

Example

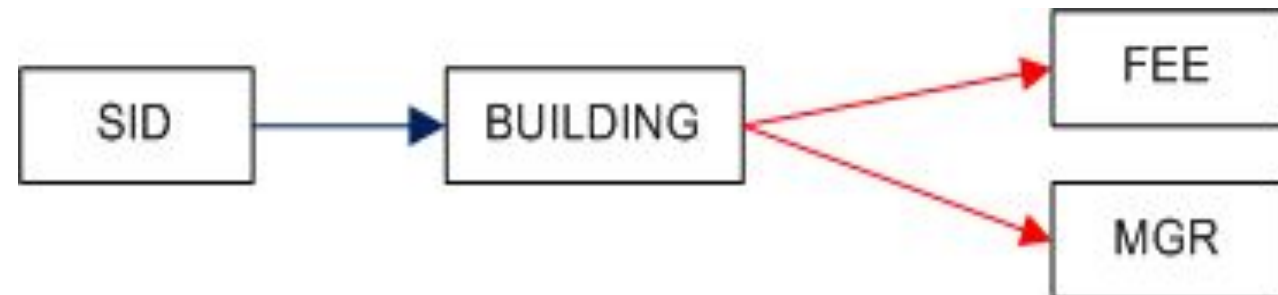
Key: { SID }

SID $\rightarrow$ Building

Building $\rightarrow$ Fee

Building $\rightarrow$ Mgr

SID	Building	Fee	Manager
100	Fenn	300	Mr. T
300	ABC	400	Ali
200	Holiday Inn	400	Tyson



Fee (and Manager) transitively depend on SID via the non-prime attribute Building. Therefore the relation is not in 3NF.

Let's try it

name	address	beer
Sally	123 Maple	Bud
Sally	123 Maple	Miller

$F = \{ \text{name} \rightarrow \text{address} \}$ Candidate key:

(name, beer)

Is this relation in 3NF?

No: *name* is not a super key,
and *address* is not a part of any candidate key.

Is this one in 3NF?

R(student, course, instructor)

$F = \{\{\text{student, course}\} \rightarrow \text{instructor},$
 $\text{instructor} \rightarrow \text{course}\}$

Candidate key: (student, course)

It is in 3NF.

Is 3NF Good Enough?

- Still have **data redundancy**
student, course, instructor
(“John Doe”, “CS2300”, “McGeehan”)
(“Bob Jones”, “CS2300”, “McGeehan”)

Caused by the FD **instructor** $\rightarrow$ **course** where instructor is not a super key.

- A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an **FD $X \twoheadrightarrow A$** holds in R, then **X is a superkey** of R

Example

- Keys: { Sid Major, Sid Fname }
- Sid Major $\twoheadrightarrow$ Fname
- Sid Fname $\twoheadrightarrow$ Major
- Fname $\twoheadrightarrow$ Major

SID	MAJOR	FNAME
100	MATH	CAUCHY
100	PHYL	PLATO
200	MATH	CAUCHY
300	PHYS	NEWTON
400	PHYS	EINSTEIN

- The relation is in 3NF but not in BCNF. Observe that Fname $\twoheadrightarrow$ Major is valid, but Fname is not a superkey.

Problem: *Student 300 drops PHYS.*

We lose information that says NEWTON is a PHYS advisor.

- A solution: (SID, FNAME), (FNAME, MAJOR)

BCNF (Boyce-Codd Normal Form)

- Each normal form is strictly stronger than the previous one
 - Every 2NF relation is in 1NF
 - Every 3NF relation is in 2NF
 - Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF
- The goal is to have each relation in BCNF (or 3NF)

Boyce-Codd Normal form

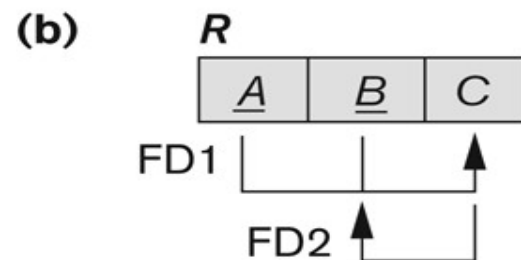
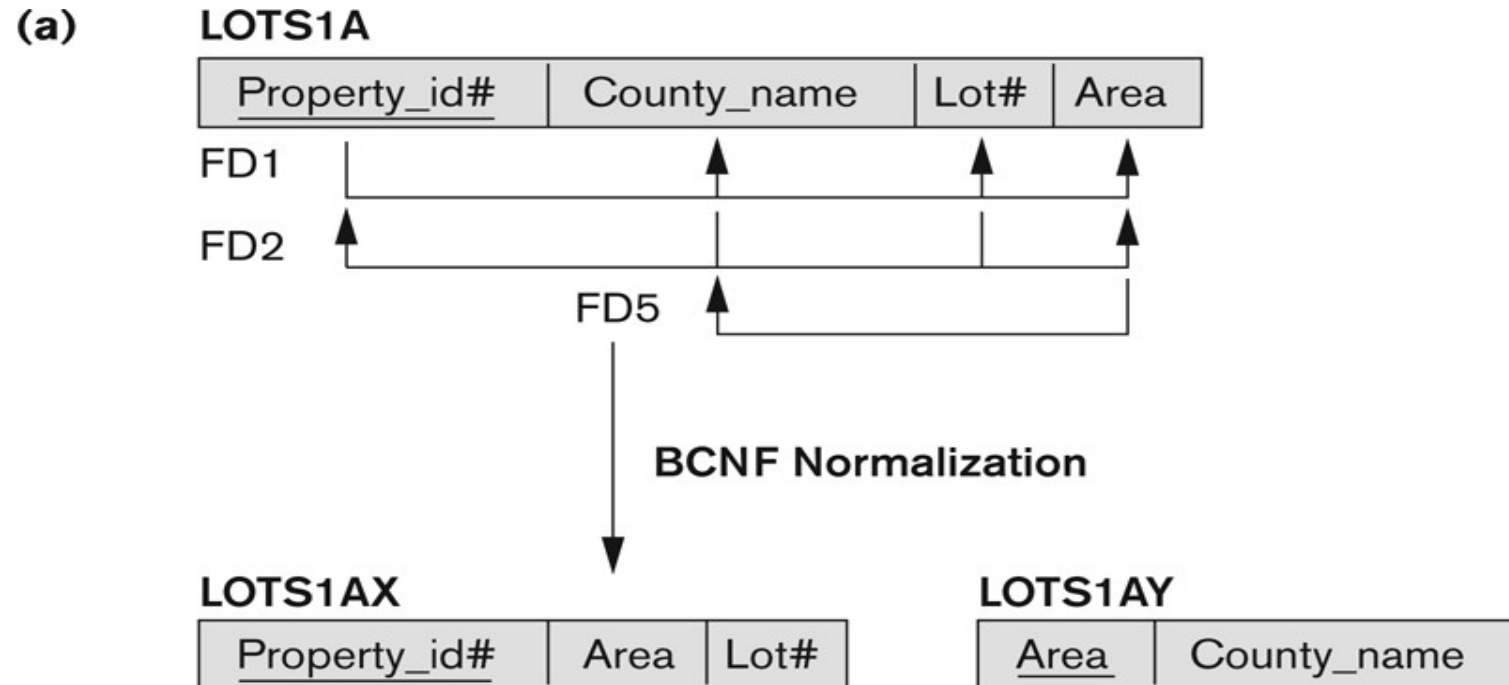


Figure 10.12

Boyce-Codd normal form. (a) BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition. (b) A schematic relation with FDs; it is in 3NF, but not in BCNF.

A relation TEACH that is in 3NF but not in BCNF

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

Key: Student Course
Student Instructor

Dependencies

Stud Course $\square$ Instructor Stud
Instructor $\square$ Course
Instructor $\square$ Course

Figure 10.13

A relation TEACH that
is in 3NF but not
BCNF.

Achieving the BCNF by Decomposition

- Three possible decompositions for relation TEACH
 - {student, instructor} and {student, course}
 - {course, instructor} and {course, student}
 - {instructor, course} and {instructor, student}
- All three decompositions will lose FD { student, course} $\square$ instructor
 - We have to settle for sacrificing the functional dependency preservation. But we cannot sacrifice the non-additivity property after decomposition.
- Out of the above three, only the 3rd decomposition will not generate spurious tuples after join.(and hence has the non-additivity property – to be discussed later) .

Thank YOU